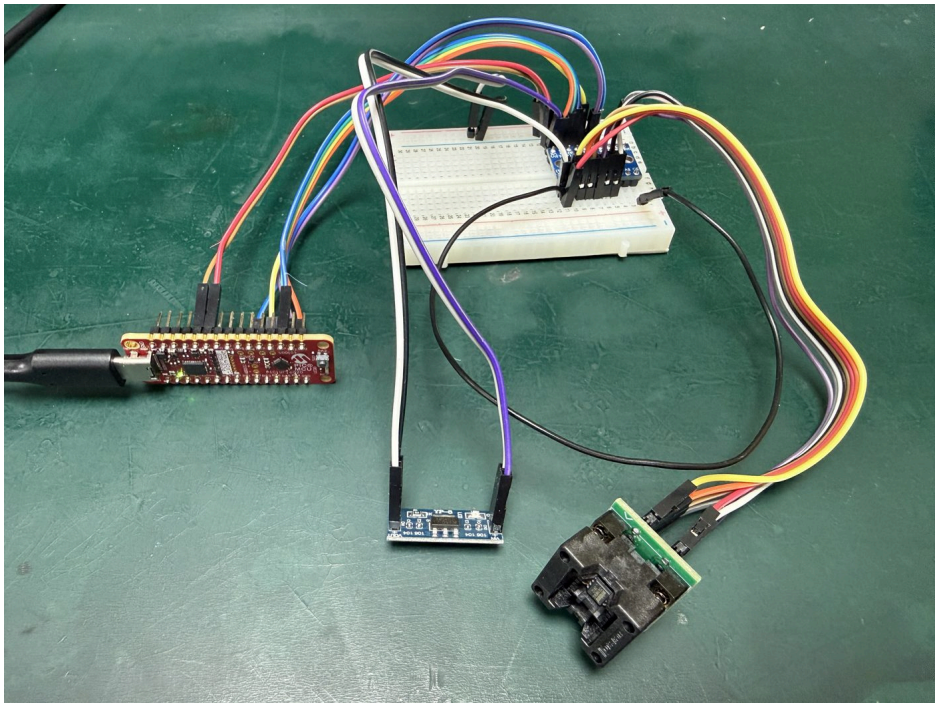
K1V2SPI Programmer

K1V2SPI Programmer User Guide

Preface

K1V2SPI Programmer reads, erases, programs and verifies **SPI NOR flash that operates at 1.2 V**. A GUI on the PC connects over USB to a PIC18F16Q20 Curiosity Nano, and the bridge firmware on the Nano talks SPI to the flash through a level translator.

Supported: Macronix MX25S1633F, GigaDevice GD25UF16E, Winbond W25Q65NE and W25Q32NE. The GUI has the same window layout on Windows and Ubuntu, and no Python installation is needed.



- **Hardware** - architecture, wiring diagrams, power and safety, bill of materials (BOM), loading the firmware into the PIC18F16Q20.
- **Protocol definition** - format, command table and error responses of the serial protocol between the PC and the Nano.
- **Using the GUI** - starting on Windows and Ubuntu, window layout, programming procedure and troubleshooting.

© 2026 K1V2SPI Programmer

Table of Contents

Preface 1

1. Introduction 3

2. Hardware 4
   - 2.1. Hardware Architecture 4
   - 2.2. Wiring Diagram 5
   - 2.3. Power and Safety 8
   - 2.4. Bill of Materials (BOM) 9
   - 2.5. Loading the Firmware into the PIC18F16Q20 10

3. Protocol Definition 12
   - 3.1. Format 12
   - 3.2. Command Table 12
   - 3.3. Error Responses 13
   - 3.4. Example Exchange 13

4. Using the GUI 15
   - 4.1. Preparing and Starting 15
   - 4.2. Window Layout 15
   - 4.3. Recommended Order for the First Use 16
   - 4.4. Normal Programming Procedure 17
   - 4.5. Buttons 17
   - 4.6. Status Register Settings (Config SR) 18
   - 4.7. Custom SPI Flash 18
   - 4.8. Troubleshooting 19

© 2026 K1V2SPI Programmer

1. Introduction

K1V2SPI Programmer reads, erases, programs and verifies **SPI NOR flash that operates at 1.2 V**. A GUI on the PC connects over USB to a PIC18F16Q20 Curiosity Nano (the "Nano" below). The bridge firmware on the Nano then talks SPI to the flash through a level translator.

Item	Description
Supported flash	Macronix MX25S1633F (16 Mbit), GigaDevice GD25UF16E (16 Mbit), Winbond W25Q65NE (64 Mbit), Winbond W25Q32NE (32 Mbit). The part is identified automatically from its JEDEC ID and JESD216H SFDP
Parts not on the list	Add them with Custom Flash in the GUI (see Section 4.7); no code change is needed
PC side	Windows 11 or Ubuntu 22.04 and later (desktop environment); no Python installation required
Connection	USB-C, virtual COM port, 500000 bps, 8N1
IO voltage protection	When the IO voltage is outside 1.14 to 1.26 V, both the firmware and the GUI refuse to program or erase

This guide is organized as: **Hardware → Protocol definition → Using the GUI**.

© 2026 K1V2SPI Programmer

2. Hardware

2.1 Hardware Architecture

The system has four parts: the GUI on the PC, the Nano (with its on-board debugger), the 1.2 V supply with the level translator, and the SPI flash.

```
[PC GUI] --USB-C (virtual COM, 500000 bps)--> [Nano: SAMD21 debugger <-UART-> PIC18F16Q20]
                                                        |  SPI, 3.3 V (VTG)
                                                        v
      [1.2 V supply MCP1826] --1.2 V--> [LSF0108PWR level translator, B side | A side]
                                                        |  SPI, 1.2 V
                                                        v
                                              [SPI NOR flash (SOIC-8 socket)]
```

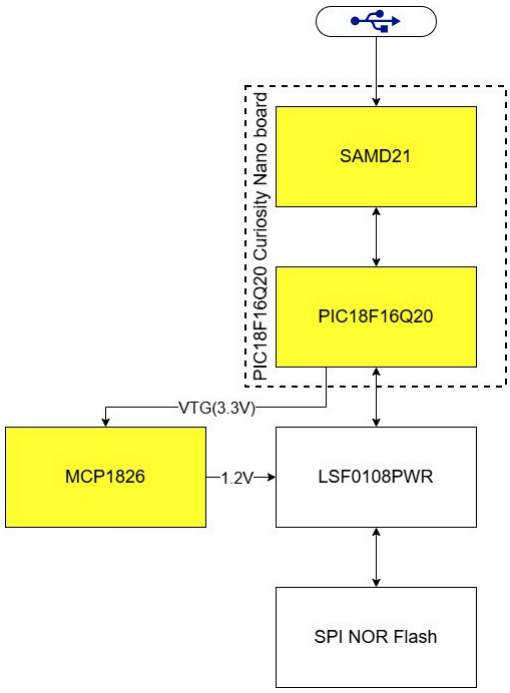


Figure 2-1. System block diagram

- **SAMD21 (the Nano's on-board debugger)**: turns the PC's USB into a virtual COM port, and also programs the PIC18F16Q20 through the MPLAB tools.
- **PIC18F16Q20**: runs the bridge firmware. It receives commands from the serial port and controls the flash with a bit-banged SPI Mode 0.
- **MCP1826**: generates 1.2 V from the Nano's VTG (3.3 V) and powers the A side of the level translator and the flash.
- **Level translator**: 3.3 V on the Nano side, 1.2 V on the flash side. **No pin of the flash may ever be connected to 3.3 V.**

© 2026 K1V2SPI Programmer

2.2 Wiring Diagram

2.2.1 Pin Assignment

Function	Nano edge position	PIC18F16Q20	Level translator B → A	Flash (SOIC-8)
CS#	Left row, SPI SS	RA4	B1 → A1	pin 1 CS#
MISO	Left row, SPI MISO	RC6	B2 → A2	pin 2 SO / SIO1
WP#	Right row, 7th hole below VTG	RB7	B3 → A3	pin 3 WP# / SIO2
MOSI	Left row, SPI MOSI	RC3	B4 → A4	pin 5 SI / SIO0
SCLK	Left row, SPI SCK	RA5	B5 → A5	pin 6 SCLK
RESET#	Left row, I2C SDA (silkscreen I3C1 SDA)	RC1	B6 → A6	pin 7 RESET# / SIO3
3.3 V	VTG	VTG	B-side reference	—
1.2 V	External 1.2 V (MCP1826)	—	A-side reference	pin 8 VCC
VSENSE (optional)	Right row, 2nd hole below VTG	RA1	—	Connect to the 1.2 V rail; the firmware measures it with the ADC
GND	GND	GND	GND	pin 4 GND

Already fixed on the board, no wiring needed: UART TX = RC4 and UART RX = RC5 (USB virtual COM port); LED0 = RC7; SW0 = RA2. RB5 / RB6 are 1.2 V MVIO pins (I2C/I3C only) and are not used by this project. Always follow the silkscreen on the Nano board when wiring.

© 2026 K1V2SPI Programmer

2.2.2 Wiring Diagram (LSF0108 + SOIC-8 flash socket, currently used)

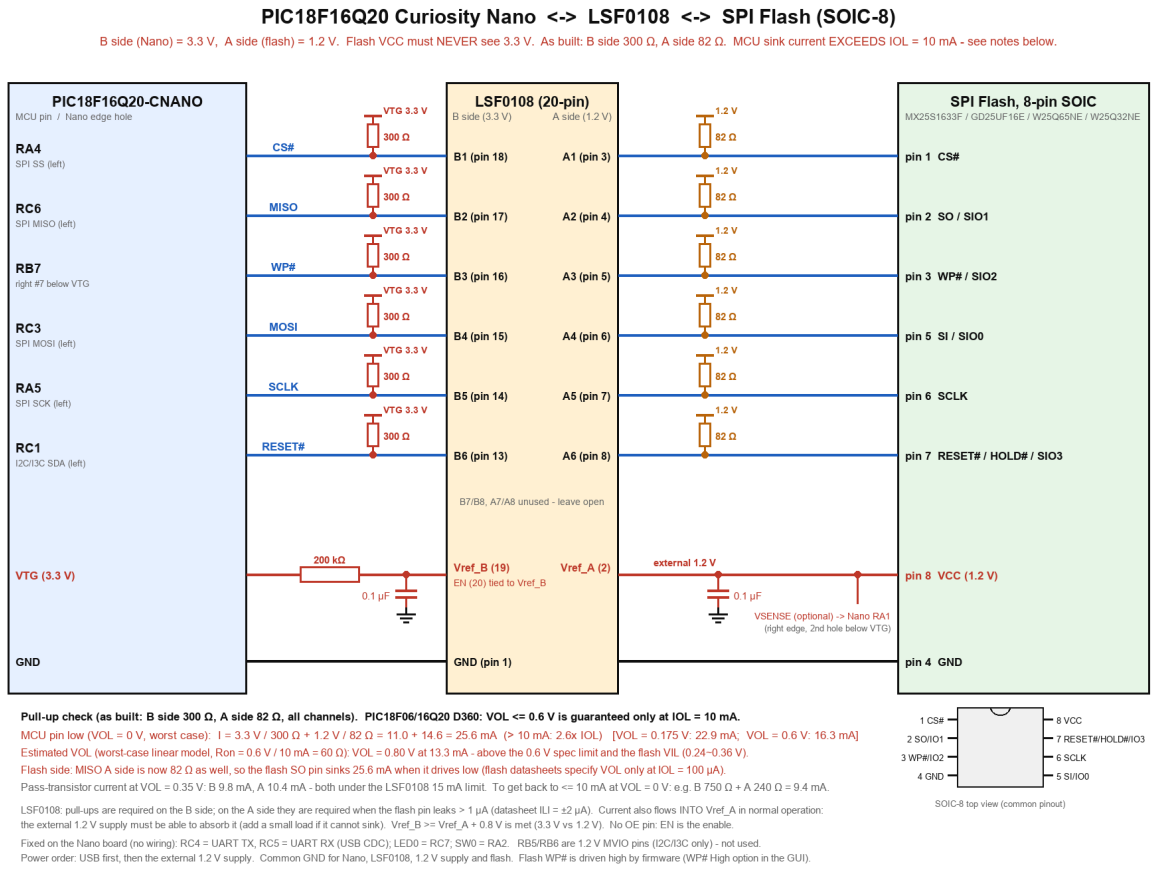


Figure 2-2. Nano ↔ LSF0108 ↔ SPI flash wiring

The LSF0108 is a switch-type (pass-gate) translator. It differs from the TXS0108E as follows:

Item	How the LSF0108 is used
Direction	Automatic bidirectional; there is no direction pin and no OE pin
Enable	EN (pin 20) is tied directly to Vref_B (pin 19) and biased from VTG through 200 kΩ; place 0.1 µF at Vref_B. EN must not be driven directly by a push-pull output
Power	Vref_B (pin 19) = VTG 3.3 V; Vref_A (pin 2) = external 1.2 V, also connected to flash pin 8; Vref_B ≥ Vref_A + 0.8 V is required
Channels	B1 to B6 (pins 18 to 13) correspond to A1 to A6 (pins 3 to 8); leave B7 / B8 and A7 / A8 open
Pull-up resistors	Required on both sides. Currently soldered: B side 300 Ω × 6, A side 82 Ω × 6 (including MISO)



Pull-up resistor limitation: When a pin is pulled low, the 300 Ω (B side) and 82 Ω (A side) pull-ups together make the MCU pin sink about 25.6 mA. This exceeds the IOL = 10 mA condition under which the datasheet guarantees VOL ≤ 0.6 V. The estimated low level is about 0.8 V, higher than the maximum VIL of the flashes (0.24 to 0.36 V). This is an estimate from the datasheets: measure the A-side low level with an oscilloscope, and treat the setup as usable only if communication is stable (all Link Test results pass in the GUI). To meet IOL ≤ 10 mA, go back to 750 Ω on the B side and 240 Ω on the A side.

© 2026 K1V2SPI Programmer

2.2.3 Alternative: TXS0108E

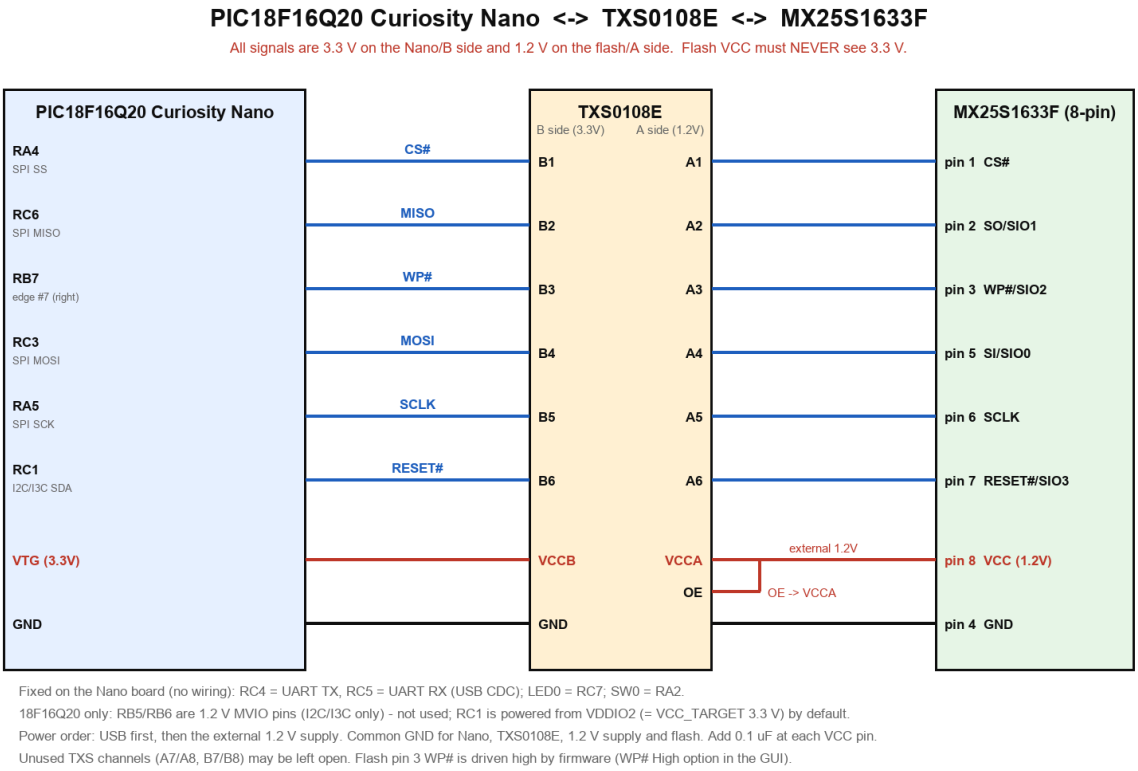


Figure 2-3. Nano ↔ TXS0108E ↔ MX25S1633F wiring

The signals and Nano pins are identical to Figure 2-2. The TXS0108E requires **VCCA ≤ VCCB**: VCCA = 1.2 V (flash side), VCCB = 3.3 V (VTG, Nano side), and **OE is tied to VCCA**. Unused channels (A7 / A8, B7 / B8) may be left open.

© 2026 K1V2SPI Programmer

2.2.4 Example of the Physical Setup

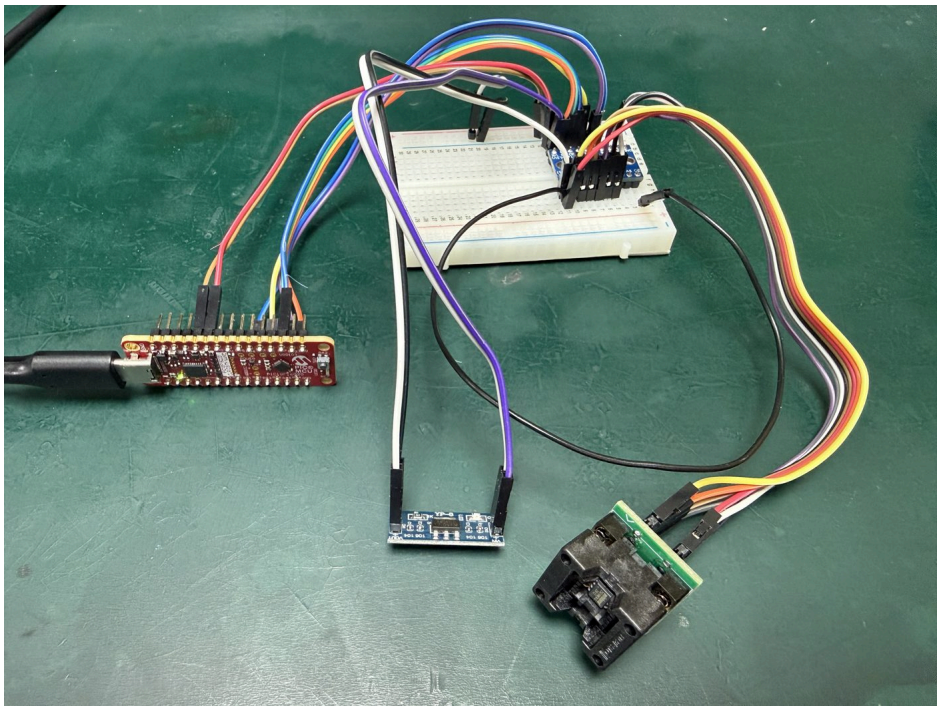


Figure 2-4. Example setup (Nano, level-translator board, 1.2 V supply module, SOIC-8 test socket)

2.3 Power and Safety

No pin of the flash may be connected to 3.3 V (or to VTG). The VCC of the MX25S1633F must stay within 1.14 to 1.6 V; the GD25UF16E, W25Q65NE and W25Q32NE must stay within 1.14 to 1.26 V. Use 1.2 V, not 1.5 V or 1.8 V.

- **Power-up order**: connect USB first (VTG comes up), then apply 1.2 V; power down in the reverse order. Connect all grounds together and place 0.1 µF next to every VCC pin.
- **Check before power-up**: use a multimeter to confirm that the A-side reference of the translator and flash pin 8 are about 1.2 V, and that no flash pin sees 3.3 V.
- **VTG voltage**: the factory default is 3.3 V. If the target voltage was ever changed with MPLAB X or a CMD file, set it back to 3.3 V first.
- **Do not** use the 1.2 V on the Nano's VDDIO3 (MCP1711, J107) to power the flash; its current and noise budget is unknown.

IO Voltage Detection (VSENSE, optional)

- The safe voltage window common to all supported flashes is **1.14 to 1.26 V**. JESD216H (SFDP) has no voltage field, so to really measure the voltage you must connect the 1.2 V rail to VSENSE (RA1).
- The firmware uses the internal FVR (2.048 V) as the ADC reference, independent of the 3.3 V VTG; the resolution is 2 mV.
- If the voltage is outside the window, the firmware refuses all writes and erases (response `ERR:vio`) and the GUI shows a red `OUT OF RANGE`.

© 2026 K1V2SPI Programmer

- If VSENSE is not connected, the GUI detects the floating pin from the difference between readings with and without the weak pull-up, falls back to ID + SFDP checking and shows a warning. You can also tick **Ignore VSENSE**.

2.4 Bill of Materials (BOM)

The table lists **all Microchip parts** on the Nano board (Altium project `PIC18F16Q20_Curiosity_Nano`, Default Assembly). The Remarks column explains what each part is used for in this programmer.

Ref.	Qty	Part number	Description	Remarks (purpose)
U100	1	PIC18F16Q20T-I/REB	8-bit MCU, 64 MHz, 64 KB flash, VQFN-20	**Bridge MCU.** Runs this project's firmware: converts UART commands into SPI timing to control the flash, measures VSENSE (the 1.2 V rail) with the ADC, and computes CRC-32
U206	1	ATSAMD21E18A-MUT	32-bit MCU, 48 MHz, 256 KB flash, QFN-32	**On-board debugger (nEDBG).** Provides the USB virtual COM port (CDC) between the PC and the UART of the PIC18F16Q20, and programs the PIC18F16Q20 (loads the bridge firmware)
U1	1	MCP1826T-ADJE/DC (based on the divider in the schematic; see note)	1 A low-dropout regulator, SOT-223-5, adjustable output	**1.2 V supply.** Generates 1.2 V from VTG (3.3 V) for the A side of the level translator and the SPI flash. The output is set by the divider R1 = 20 kΩ, R2 = 10 kΩ: 0.41 V × (20 k + 10 k) / 10 k ≈ 1.23 V
U200	1	MIC5353YMT-TR	500 mA adjustable-output LDO, MLF-6	**Target voltage (VTG) regulator.** The debugger adjusts its feedback voltage to produce the target voltage used by the PIC18F16Q20 and the 3.3 V-side signals (factory default 3.3 V, limited to 1.8 to 3.6 V on this board). The MCP1826 and the 3.3 V side of the level translator are both powered from VTG
U201	1	MIC94163YCS-TR	High-side load switch, 3 A, 6-WLCSP	**Target power switch.** Sits between the target regulator U200 and the VTG output (VCC_TARGET); the debugger controls whether VTG is delivered. When the VOFF pin is pulled low the debugger switches the target regulator off, and only then may VTG be powered externally. Its minimum output voltage of 1.70 V limits the lowest target voltage
U202	1	MIC2008YML-TR	Power switch, 5.5 V, 2.1 A, VDFN-6	**USB 5 V input protection.** Delays power-up, limits the VBUS slew rate and limits the current to 500 mA to protect the USB port
U203	1	MIC5528-3.3YMT-TR	3.3 V fixed-output LDO, 6-TDFN	**Debugger supply.** Provides 3.3 V to the SAMD21 (U206) and its peripherals

© 2026 K1V2SPI Programmer

U101	1	MCP1711T-12I/5X	1.2 V fixed-output LDO, UQFN-4	**VDDIO3 supply of the PIC18F16Q20** (the 1.2 V MVIO domain of RB5 / RB6). This project does not use those two pins; do not use it to power the flash

Note: the Altium BOM export lists U1 as `MCP1826T-1802E/DC` (fixed 1.8 V version), but in the schematic U1 sets its output with an R1 / R2 divider, which is how the adjustable MCP1826 is wired. According to the part-number rules in the MCP1826 datasheet, the adjustable version is written `MCP1826T-ADJE/DC` (T = tape and reel, ADJ = adjustable output, E = −40 to +125 °C, DC = SOT-223-5). For purchasing and assembly, use the part number actually fitted.

2.5 Loading the Firmware into the PIC18F16Q20

The first time you use the programmer, load the bridge firmware `K1V2SPI_FW_PIC18F16Q20.hex` into the PIC18F16Q20. The file is in `firmware/prebuilt/`. While programming, **connect only the USB-C cable; do not connect the flash or the 1.2 V supply**.

Method 1: MPLAB IPE (recommended, verifies by read-back)

1. Install MPLAB X IDE (v6.30 or later); it contains the IPE and `ipecmd.exe`.
2. Connect the Nano to the PC with a USB-C cable that supports data transfer.
3. Run in a command prompt:

```
firmware\flash_nano.bat
```

4. The script programs the board through the on-board debugger (nEDBG) and reads back what was written for a bit-by-bit comparison. Programming is successful only when the last line reads `Programmed and verified`.

If MPLAB X is installed in another location, set the environment variable `IPECMD` to the path of `ipecmd.exe` first.

Method 2: MPLAB IPE graphical interface

1. Open MPLAB IPE, select `PIC18F16Q20` as the Device and the Nano's on-board debugger (nEDBG) as the Tool.
2. Click Connect, choose `K1V2SPI_FW_PIC18F16Q20.hex` as the Hex file, and click Program.
3. When programming finishes, click Verify to confirm.



Dragging the HEX file onto the CURIOSITY drive is not recommended. On the PIC18F16Q20-CNANO, a large HEX file was once truncated (program memory from 0x1998 upward and the configuration bits were left blank) while `STATUS.TXT` still showed Ready. The script's `--dragdrop` option is only a fallback when the IPE cannot be found, and it does not verify.

Confirming that the firmware is loaded correctly

1. Open the GUI (see Chapter 4), select the Nano's serial port and click **Connect**.
2. The Log shows `Bridge alive.` and the version line shows `Bridge: PIC18F16Q20-CNANO firmware 5.1.1`; the firmware is working.

© 2026 K1V2SPI Programmer

3. No flash is connected yet, so `ID:FF FF FF` is normal at this point.

Building the Firmware Yourself (optional)

This requires MPLAB XC8 v3.x and PIC18F-Q_DFP. Run `firmware\build.bat` to produce `firmware\prebuilt\K1V2SPI_FW_PIC18F16Q20.hex`. The tool locations can be overridden with the environment variables `XC8_BIN` and `DFP_18FQ`.

© 2026 K1V2SPI Programmer

3. Protocol Definition

The GUI and the Nano communicate over the USB virtual COM port at **500000 bps, 8N1** (500 kbps is the upper limit of the Nano debugger's CDC).

3.1 Format

- A command is **one line of ASCII ending with** `\n`. Command names must be in **upper case** (for example `PING`), and the command and its arguments are separated by spaces.
- All numbers are **hexadecimal**, without `0x`, and the letters A to F must be **upper case** (for example `PP 1F00 100`).
- A response is one line, `OK…` or `ERR:xxx`; read-type commands send raw bytes after the `OK`.
- Addresses are 24 bits (maximum 0xFFFFFF), which covers the 8 MB of the W25Q65NE.

3.2 Command Table

Command	Response	Description
`PING`	`OK PONG`	Communication test
`INFO`	`OK PIC18F16Q20-CNANO SPINOR-BRIDGE 5.1.1`	Board name and firmware version
`ID`	`ID:C2 2B 25`	JEDEC RDID (9Fh). MX25S1633F = `C2 2B 25`, GD25UF16E = `C8 83 15`, W25Q65NE = `EF 65 17`, W25Q32NE = `EF 65 16`
`SFDP addr len`	`OK` followed by len bytes	JESD216 Read SFDP (5Ah + 3-byte address + 8 dummy clocks), len ≤ 256
`VIO`	`VIO:aaaa bbbb`	VSENSE (RA1) voltage in mV (hexadecimal). aaaa = normal reading, bbbb = reading with the weak pull-up enabled; a large difference means the pin is floating (not connected)
`VCHK 0|1`	`OK`	1 = enable the voltage interlock: when VSENSE is outside 1.14 to 1.26 V, PP / PW / SE / B32 / BE / CE / WRSR always return `ERR:vio`. **The power-up default is 1**
`LINK n`	`LINK:gggg/nnnn`	Reads the ID n times in a row and reports how many times it matched the first reading with a vendor code other than 00 / FF (used by Link Test)
`RDSR` / `RDCR` / `RDSR2`	`SR:xx` / `CR:xx` / `SR2:xx`	Read Status Register 1 (05h) / Configuration Register (15h, MX25) / Status Register 2 (35h, GD25UF16E and Winbond)
`WRSR xx [yy]`	`OK`	WREN + WRSR + wait for completion. When a second byte yy is given it is written to SR2 as well (writing only 1 byte on the GD25UF16E clears the writable bits of SR2)
`READ addr len`	`OK` followed by len bytes	Reads the flash, len ≤ 0x10000
`CRC addr len`	`CRC:xxxxxxxx`	The Nano computes the CRC-32 (same as zlib) of a flash range; used for verification and for the Read→File double-check
`PP addr len`	`RDY` → PC sends len bytes → `OK`	Single-page write, len ≤ 256, must not cross a page boundary

© 2026 K1V2SPI Programmer

`PW addr len`	(no RDY) data follows right after the command line → `OK`	Pipelined write: the Nano returns `OK` as soon as it has received the data and started the page write; the next `PW` / `SYNC` waits for the previous page to finish. A UART overrun returns `ERR:ovf`
`SYNC`	`OK`	Waits until the current write / erase is complete
`SE addr`	`OK`	4 KB Sector Erase (20h)
`B32 addr`	`OK`	32 KB Block Erase (52h)
`BE addr`	`OK`	64 KB Block Erase (D8h)
`CE`	`OK`	Chip Erase (60h); waits up to 150 s
`WP 0|1`	`OK`	Drives the WP# pin (RB7)
`SPEED n`	`OK`	SPI delay from 0 (fastest) to 3F; a larger number is slower
`SLEW 0|1`	`OK`	1 = slew-rate-limit the SPI output pins (slower edges)
`LED 0|1`	`OK`	1 = LED0 on (RC7)
`HWRESET`	`OK`	Pulls the RESET# pin low for 2 ms (RC1)
`RST`	`OK`	Software reset: RSTEN (66h) + RST (99h)
`RDP`	`OK`	Release from Deep Power-Down (ABh) + wait 50 µs + one RDSR. The firmware runs it once at boot; the GUI also sends it before reading the ID in Auto Detect

3.3 Error Responses

Response	Meaning
`ERR:cmd`	Unknown command
`ERR:arg`	Invalid argument
`ERR:busy`	The flash is busy
`ERR:wel`	The write is protected, or Write Enable did not take effect
`ERR:timeout`	Timed out while waiting
`ERR:ovf`	UART receive overrun
`ERR:vio`	VSENSE is outside 1.14 to 1.26 V; the write or erase was refused

3.4 Example Exchange

```
PC   -> PING
Nano -> OK PONG
PC   -> ID
Nano -> ID:C2 2B 25
PC   -> SE 0
Nano -> OK
PC   -> PP 0 100            (write 0x100 = 256 bytes at address 0; numbers are hexadecimal)
Nano -> RDY
PC   -> [256 bytes of raw data]
Nano -> OK
PC   -> CRC 0 100
Nano -> CRC:1A2B3C4D
```


© 2026 K1V2SPI Programmer

The **Raw Monitor** in the GUI can send the commands above directly and show the raw responses; it is useful for debugging.

© 2026 K1V2SPI Programmer

4. Using the GUI

4.1 Preparing and Starting

The Windows and Ubuntu versions have exactly the same window layout and the same functions; they differ only in how they are started and in the serial port names.

Item	Windows	Ubuntu
Requirements	Windows 10 / 11	Ubuntu 22.04 or later, desktop environment (X11 or Wayland)
Executable	`gui\Win\dist\K1V2SPI_Programmer.exe`	`gui/Linux/dist/K1V2SPI_Programmer`
Installation	None; double-click to run	None. The first time, make the file executable: `chmod +x K1V2SPI_Programmer`
Starting	Double-click the exe	Run `./K1V2SPI_Programmer` in a terminal, or double-click it in the file manager
Serial port name	Something like `COM3`, described as Curiosity Virtual COM Port	Something like `/dev/ttyACM0`
Serial port permission	No setup needed	The first time, add yourself to the `dialout` group and then **log out and log in again**: `sudo usermod -aG dialout $USER`
Settings and history folder	`%APPDATA%\MX25S1633F_Programmer\`	`~/.config/MX25S1633F_Programmer/`
Antivirus software	A single-file exe is occasionally flagged by mistake; you can run from source instead (see Section 4.8)	—

Note: The GUI and other serial-port programs **cannot open the same serial port at the same time**. After it starts, if the PC has **exactly one** Microchip (VID 03EB) serial port, the GUI selects it and connects automatically.

4.2 Window Layout

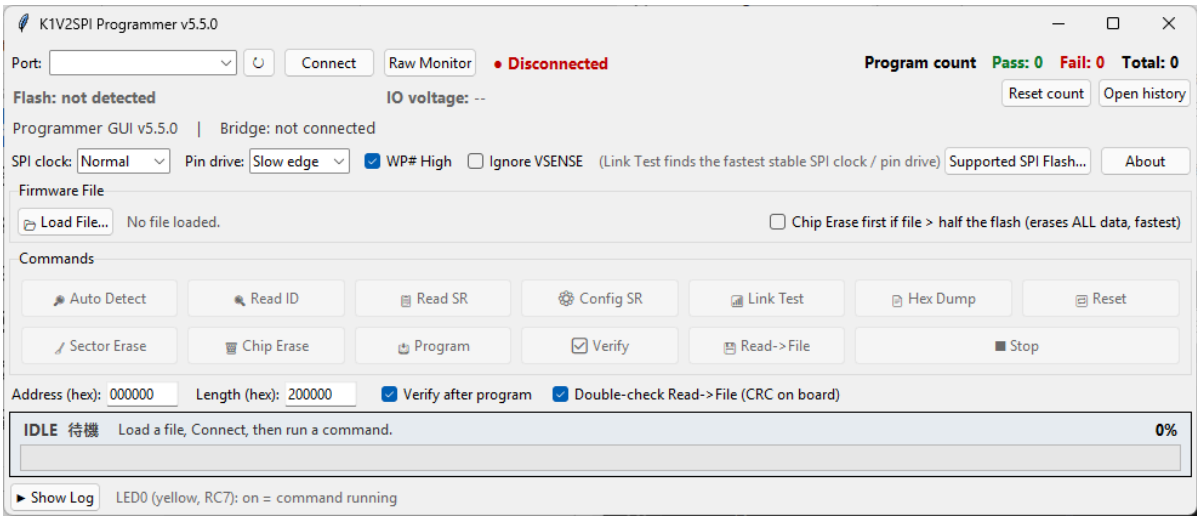


Figure 4-1. Main window, Windows version

© 2026 K1V2SPI Programmer

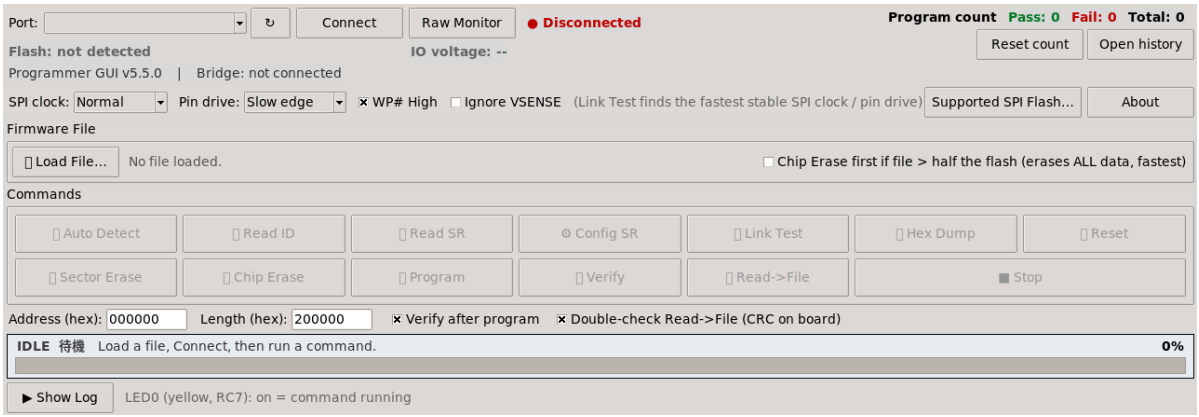


Figure 4-2. Main window, Ubuntu version

Area	Contents
Top row	**Port** drop-down (finds the Curiosity Virtual COM Port automatically), ↻ refresh, **Connect**, **Raw Monitor**, connection status (● Disconnected / Connected)
Status area	**Flash** (the part identified by Auto Detect), **IO voltage** (the voltage VSENSE measured), programmer version and Nano firmware version
Program count	Top right: cumulative programming results **Pass (green) / Fail (red) / Total**; **Reset count** clears them and **Open history** opens the programming history (CSV: time, result, file name, CRC, seconds, failure reason)
SPI clock row	**SPI clock** (Max to Slowest), **Pin drive** (Slow edge / Fast edge), **WP# High**, **Ignore VSENSE**, **Supported SPI Flash…**, **About**
Firmware File	**Load File…** loads a .bin or .hex file and shows its size, address range and CRC32; on the right is the **Chip Erase first** option
Commands	13 command buttons in two rows (see Section 4.5); Stop aborts at any time
Parameter row	**Address (hex)**, **Length (hex)**, **Verify after program**, **Double-check Read->File**
Progress area	Current state (IDLE, RUNNING, ERASING, WRITING, VERIFYING, DONE, STOPPED, FAILED), a message and a 0 to 100 % progress bar
Show Log	Collapsed by default. Click ▶ **Show Log** to expand and ▼ **Hide Log** to collapse; if an error occurs while collapsed, the button shows "⚠ new error"

Small windows open to the right of the main window: **Supported SPI Flash** (the list of supported parts; the detected part is highlighted; the buttons at the bottom are Custom Flash…, Import… and Export…), **Custom Flash**, **Config SR**, **Raw Monitor** and **About** (program name, GUI version, update date, MCU, firmware version, designer and GitHub URL).

4.3 Recommended Order for the First Use

1. **Without the flash first**: connect only the USB-C cable and load the firmware as described in Section 2.5. Open the GUI and click Connect; `Bridge alive.` means communication works (`ID:FF FF FF` is normal at this point).
2. **Wire with the power off**: wire the level translator and the 1.2 V supply as described in Section 2.2, but do not insert the flash yet. After power-up, use a multimeter to confirm the 1.2 V rail and flash socket pin 8 are about 1.2 V and that no socket pin has 3.3 V.

© 2026 K1V2SPI Programmer

3. **Insert the flash**: set SPI clock to Slow and click Connect. `Detected Macronix MX25S1633F …` should appear (the matching part name is shown for other chips).
4. Once communication is stable, click **Link Test** so the program finds the fastest stable SPI clock and Pin drive and applies them automatically.

4.4 Normal Programming Procedure

1. **Connect**: the Log shows `Bridge alive.` and `Detected <vendor> <part> …` (Auto Detect runs automatically on Connect).
2. (Recommended) Back up first: enter `000000` as Address and use the whole capacity filled in by Auto Detect as Length (for example 2 MB = `200000`), then click **Read->File**.
3. Click **Load File…** and choose the file to program. A .hex file sets the start address automatically; a .bin file starts at 0x000000 by default.
4. Check Address, then click **Program**. The program first erases the blocks it needs and then writes page by page.
5. With **Verify after program** ticked, the data is verified automatically after programming; the Log ends with `VERIFY OK` and the Pass count increases by 1.

Speed design: erasing automatically chooses the largest aligned block (64 KB → 32 KB → 4 KB); page writes are pipelined and completely blank pages (all 0xFF) are skipped; verification compares a CRC computed by the Nano for every 64 KB, so no data needs to be read back to the PC. When the file is larger than half of the chip, you can tick **Chip Erase first** (erases the whole chip, fastest).

4.5 Buttons

Button	Action	Parameters used
Auto Detect	Checks the IO voltage (VSENSE) → reads the JEDEC ID (9Fh) → reads SFDP (5Ah) to identify vendor, part and capacity. If the voltage is not 1.2 V or identification fails, Program / Erase / Config SR are locked. For an ID that is not on the list, the specification is listed from the SFDP and the label turns orange "SFDP only"; only Hex Dump / Read->File are available	—
Read ID	Reads the JEDEC ID	—
Read SR	Reads and decodes the Status Register (bit names follow the detected part; the GD25UF16E and Winbond parts also show SR2)	—
Config SR	Opens the dialog to change the writable SR1 bits (see Section 4.6)	—
Link Test	Runs 200 RDID reads at each SPI clock, finds the fastest stable SPI clock and Pin drive and applies them	—
Hex Dump	Shows the hex contents of the given range in the Log (up to 4 KB)	Address, Length
Reset	Software-resets the flash (66h + 99h)	—
Sector Erase	Erases the 4 KB sector that contains Address, then reads it back to check it is all FF (a protected sector is reported as an error instead of a false success)	Address
Chip Erase	Erases the whole flash (asks for confirmation first). SR1/SR2 are read first: if BP / CMP protection is active the erase is refused with the reason (the chip itself ignores Chip Erase silently, without an error). Afterwards a full blank check with the board CRC confirms the whole chip is FF before success is reported	—

© 2026 K1V2SPI Programmer

Program	Erases the needed sectors → writes page by page → (optional) verifies	Address
Verify	Compares the flash with the loaded file	Address
Read->File	Reads the flash and saves it as a .bin file; with Double-check ticked, the data is checked once more with the Nano's CRC	Address, Length
Stop	Aborts the running action	—

SPI clock / Pin drive: when the connection is unstable, set SPI clock to Slow and Pin drive to Slow edge (slew-rate limited; the TXS0108E and LSF0108 tend to ring with fast edges), then use Link Test to find the fastest setting. **WP# High** is ticked by default and keeps WP# high (the WP# wire must be connected).

4.6 Status Register Settings (Config SR)

The bit definitions of SR1 (05h) are **different for the three vendors**; the GUI switches automatically according to the detected part:

Bit	MX25S1633F	GD25UF16E	W25Q65NE / W25Q32NE
7	SRWD: SR write protect	SRP0: SR protect	SRP: SR protect
6	QE: Quad Enable	BP4	SEC: protect unit 4 KB / 64 KB
5	BP3	BP3	TB: protect from top / bottom
4 to 2	BP2 to BP0	BP2 to BP0	BP2 to BP0
1	WEL (read-only)	WEL (read-only)	WEL (read-only)
0	WIP (read-only)	WIP (read-only)	BUSY (read-only)

- **Chip Erase does not run unless all BP bits are 0**, and protected areas cannot be erased or programmed. If programming fails, first go to Config SR and clear the BP bits to 0.
- When you tick bits, the SR value is shown in real time at the top right; **Default** clears everything to 0; **OK** performs WREN → WRSR → read-back comparison.
- If SRWD / SRP0 / SRP = 1 prevents writing: first tick **WP# High** (the WP# wire must be connected), then change the SR. SRP1 = 1 on the GD25UF16E and SRL = 1 on Winbond are power-cycle locks and are released only by a **power cycle**.
- LB3 to LB1 of SR2 are OTP bits (they cannot be undone once set to 1); the GUI does not allow changing them.
- **The W25Q64NE is not supported**: its JEDEC ID (`EF 65 17`) and capacity (64 Mbit) are the same as the W25Q65NE, so they cannot be told apart; when inserted it is shown as a W25Q65NE.

4.7 Custom SPI Flash

When a part is not on the list, the Log shows `Blocked: … not in the supported list`. Click **Supported SPI Flash…** → **Custom Flash…** and fill in the parameters; no code change, repackaging or reprogramming of the Nano is needed. Capacity, page size, erase types and address mode are all read from the chip's SFDP, so you only need to enter what SFDP does not provide:

Field	Required	Description

© 2026 K1V2SPI Programmer

Vendor / Part	Yes	Display name
JEDEC ID	Yes	The 3 bytes returned by 9Fh, such as `EF 65 18` ; must not be the same as a built-in part
Chip erase max (s)	Yes	The maximum tCE from the datasheet, 1 to 150 seconds; a value that is too small times out before the erase finishes
Density (Mbit)	No	If given, it is compared with the SFDP capacity and programming is blocked on a mismatch (protects against a wrong part behind the same ID)
SR1 writable bits	No	The bits shown in Config SR, in the form `NAME=bit` ; default `BP2=4, BP1=3, BP0=2`
VCC min-max	No	Must cover 1.14 to 1.26 V, otherwise saving is refused
Has Status Register 2	No	Winbond / GigaDevice type: WRSR must write SR2 back together with SR1

Procedure: connect the chip and click Auto Detect → Custom Flash… → **Use detected flash** to fill in the ID and capacity → add Chip erase and the other fields from the datasheet → **Save / Update** → click Auto Detect again. The settings are stored in `custom_flash.json` (if the executable has this file next to it, that file is used; otherwise it is in the settings folder listed in Section 4.1).

Import / Export: the **Supported SPI Flash** window has **Import…** and **Export…** buttons at the bottom. They work on **all** custom parts at once, not only the selected one. **Export…** saves every custom part to one JSON file (default name `custom_flash.json`). **Import…** reads such a file and merges it into the list: an entry with the same JEDEC ID is updated and all other existing entries are kept. Entries with an invalid field or with the ID of a built-in part are skipped, and the reason is shown in the Log. Use this to move your settings to another computer.



Caution: The erase time and the SR mask of a custom part are supplied by you, and the program cannot check them against the datasheet. **For the first run, test with unimportant data** and make sure tCE is not set too small.
Limits: only 3-byte addressing (capacity ≤ 128 Mbit), 256 B pages, 4 KB erase (20h), tCE ≤ 150 s and the 1.14 to 1.26 V IO voltage window are supported; parts without SFDP cannot be programmed.

4.8 Troubleshooting

Symptom	Possible cause and remedy
No serial port found	Use a USB-C cable that supports data transfer; click ↻ to refresh. On Windows, check Device Manager for a Curiosity Virtual COM Port; on Ubuntu, make sure you are in the `dialout` group and have logged in again, and that no other program is using `/dev/ttyACM*`
`PING … ERR:timeout`	Another program is using the serial port; unplug and replug the Nano's USB cable and reconnect; look at the raw output in Raw Monitor
The GUI cannot connect after the firmware was loaded	Unplug and replug USB and click ↻ to refresh; the baud rate is 500000
`ID:FF FF FF`	MISO is always high: the flash has no power (check 1.2 V), the translator is not enabled, or MISO is not connected properly

© 2026 K1V2SPI Programmer

`ID:00 00 00`	MISO is always low: a wiring short, no power to the flash, or the flash is inserted the wrong way round (check the pin 1 position)
The ID is occasionally wrong	Click Link Test; set SPI clock to Slow and Pin drive to Slow edge; shorten the jumper wires; add 0.1 µF capacitors
Erase / Program fails	BP bits are protecting the flash → clear BP to 0 in Config SR
WRSR fails	SRWD / SRP0 / SRP = 1 with WP# low → tick WP# High; GD25UF16E SRP1 = 1 or Winbond SRL = 1 → power-cycle
`ERR:vio` / `OUT OF RANGE`	The 1.2 V rail measured by VSENSE is outside 1.14 to 1.26 V and the firmware refuses writes and erases. Check the 1.2 V supply with a multimeter
`Blocked: ID … not in the supported list`	The part is not on the list: the Log lists its specification from SFDP and only reading is possible. To program it, add it with Custom Flash (Section 4.7)
`Blocked: SFDP density … does not match`	The ID does not match the SFDP capacity; most likely the part is not the expected model. Check the marking on the chip
Verify MISMATCH	The area was not erased before programming, or the SPI speed is too high (lower the speed or run Link Test)
The Windows exe is blocked by antivirus software	A single-file PyInstaller exe is occasionally flagged by mistake; install Python 3.9 or later and pyserial, then run `python gui\App_GUI.py`

© 2026 K1V2SPI Programmer